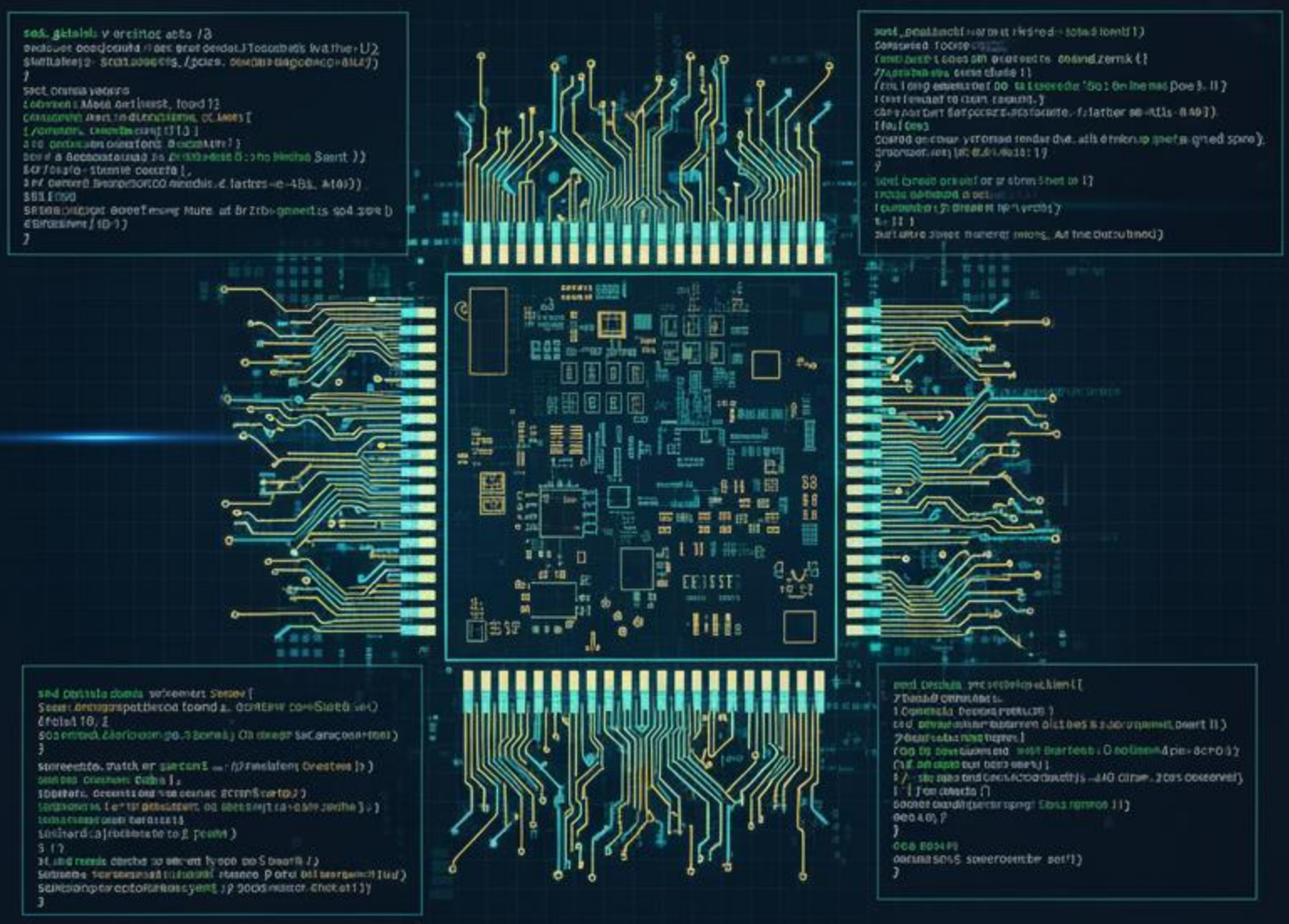
TCL SCRIPTING FOR VLSI PHYSICAL DESIGN ENGINEERS

From Basics to Advanced



Bapuji Banothu | M.Tech VLSI SD

OpenROAD Yosys Cadense

Tcl Scripting for VLSI Physical Design Engineers

From Basics to Advanced

A Comprehensive Training Manual
For Students, ASIC Engineers & VLSI PD Engineers

Author: Bapuji Banothu
M.Tech – VLSI System Design
© 2026 – All Rights Reserved

Covering: OpenLane · OpenROAD · Yosys · Cadence Innovus · Synopsys ICC2 · PrimeTime

1. Introduction 6
1.1 What is Tcl? 6
1.2 Why Tcl in ASIC Tools? 6
1.3 Where is Tcl Used in VLSI Physical Design? 6
2. Running Tcl Scripts 7
2.1 Interactive Mode – tclsh 7
2.2 Running a Script File 7
2.3 Running Tcl Inside EDA Tools 7
2.4 Shebang Line for Direct Execution 7
3. Tcl Basics 9
3.1 Variables 9
3.2 Expressions 9
3.3 Comments 9
3.4 The puts Command 10
3.5 Substitution Rules 10
4. Control Structures 11
4.1 if Statement 11
4.2 if-else Statement 11
4.3 if-elseif-else Chain 11
4.4 switch Statement 11
4.5 Real VLSI Example – Checking PDK 12
5. Loops 13
5.1 for Loop 13
5.2 foreach Loop 13
5.3 while Loop 13
5.4 Loop Control: break and continue 14
6. Procedures (procs) 15
6.1 Defining a Procedure 15
6.2 Procedures with Default Arguments 15
6.3 Variable Scope – global and upvar 15
6.4 Real VLSI Procedure Examples 15
7. Lists and Arrays 17
7.1 Creating Lists 17
7.2 List Operations 17
7.3 List Iteration 17
7.4 Arrays 18

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

7.5 Dictionaries (Tcl 8.5+) 18
8. String Operations 19
8.1 String Length and Index 19
8.2 Case Conversion 19
8.3 String Matching (glob patterns) 19
8.4 String Search 19
8.5 String Replace and Trim 19
8.6 format – Formatted Output 20
9. File Handling 21
9.1 Opening and Closing Files 21
9.2 Reading Line by Line 21
9.3 Parsing a Synthesis Report 21
9.4 Writing Reports 22
10. Directory and File Operations 23
10.1 File Tests 23
10.2 Creating Directories 23
10.3 Collecting Files with glob 23
10.4 File Copy, Move, Delete 24
11. Environment Variables 25
11.1 Accessing Environment Variables 25
11.2 PDK and Tool Configuration 25
11.3 Setting Environment Variables from Tcl 25
11.4 OpenLane Configuration via env 25
12. Regular Expressions 27
12.1 regexp – Pattern Matching 27
12.2 Extracting Timing Violations 27
12.3 regsub – Substitution 27
12.4 Common Regex Patterns for VLSI 28
13. Tcl in ASIC Design Flows 29
13.1 Synthesis with Yosys 29
13.2 OpenROAD Physical Design Flow 29
13.3 STA with PrimeTime (Synopsys) 30
13.4 Cadence Innovus Flow 30
14. Practical VLSI Tcl Scripts 32
14.1 RTL File Collector 32
14.2 Synthesis Log Parser 32

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

14.3 Timing Report Analyzer 33
14.4 Automatic Design Folder Creator 33
14.5 OpenLane Automation Script 34
15. Advanced Tcl Concepts 36
15.1 Namespaces 36
15.2 Error Handling with catch 36
15.3 Packages 37
15.4 exec – Running System Commands 37
15.5 Callbacks and apply 37
16. Example Mini Projects 38
16.1 Project 1 – Verilog Module Scanner 38
16.2 Project 2 – Design File Checker 38
16.3 Project 3 – Simple Design Report Generator 39
17. Best Practices for VLSI Tcl Scripts 41
17.1 Script Structure 41
17.2 Naming Conventions 41
17.3 Debugging Techniques 41
17.4 Performance Tips 42
18. Interview Questions – Tcl for VLSI Engineers 43
Q1. What is Tcl and why is it used in EDA tools? 43
Q2. What is the difference between {} and "" in Tcl? 43
Q3. How do you read a file line-by-line in Tcl? 43
Q4. What is the difference between set and variable? 43
Q5. How does catch work in Tcl? 43
Q6. What is lsearch and how is it used? 43
Q7. How do you pass a variable by reference in Tcl? 43
Q8. What does the glob command do? 44
Q9. How do you extract a number from a string using regexp? 44
Q10. What is the difference between expr and set? 44
Q11. How do you define and call a procedure with default arguments? 44
Q12. How do you use Tcl arrays in VLSI scripts? 44
Q13. What is the purpose of namespaces in Tcl? 44
Q14. How do you write to a file in Tcl? 45
Q15. What is the incr command? 45
Q16. What is source in Tcl? 45
Q17. Explain the difference between lindex and lrange 45

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

Q18. What is format used for in Tcl? ... 45
Q19. How do you check if a variable is defined? ... 45
Q20. How does Tcl handle error propagation in ASIC flows? ... 45
19. Conclusion ... 47
19.1 What You Have Learned ... 47
19.2 The ASIC Flow Automation Advantage ... 47
19.3 Key Takeaways ... 47
19.4 Next Steps ... 47

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

1. Introduction

1.1 What is Tcl?

Tcl (Tool Command Language) is a powerful, lightweight scripting language created by John Ousterhout in 1988. It was originally designed to be embedded into other applications as their scripting engine. Tcl is known for its simplicity, portability, and excellent string-processing capabilities.

Unlike compiled languages such as C or C++, Tcl scripts are interpreted at runtime, making them ideal for automation, configuration, and rapid prototyping tasks. The language follows a simple rule: everything is a string, and every operation is a command.

1.2 Why Tcl in ASIC Tools?

Tcl has become the de facto scripting language for Electronic Design Automation (EDA) tools. Here is why:

- Simple and embeddable – EDA vendors can embed Tcl directly into their C/C++ tool engines
- Cross-platform – runs identically on Linux, Windows, and macOS
- Excellent string processing – crucial for parsing log files and reports
- Interactive and scriptable – designers can type commands at a prompt or run scripts
- Industry-standard – all major EDA vendors adopted Tcl as their scripting interface

1.3 Where is Tcl Used in VLSI Physical Design?

EDA Tool / Flow	How Tcl is Used
OpenLane	Flow configuration (config.tcl), all flow commands
OpenROAD	Floor planning, placement, CTS, routing commands
Yosys	RTL synthesis scripts and commands
Cadence Innovus	Full PD flow scripting from import to sign-off
Synopsys ICC2	Full PD flow: placement, routing, ECO
PrimeTime (STA)	Timing constraints, reporting, ECO analysis
Calibre (DRC/LVS)	Rule file scripting and automation
Perl/Python	Integration glue scripts calling Tcl command lines

Throughout this document, you will learn Tcl step-by-step with examples that are directly applicable to real ASIC design flows.

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

2. Running Tcl Scripts

2.1 Interactive Mode – tclsh

The simplest way to run Tcl is to launch the interactive interpreter called tclsh (Tcl Shell). Type commands and get immediate results.

```
# Start the Tcl shell
$ tclsh

# You will see the % prompt
% puts "Hello, VLSI World!"
Hello, VLSI World!

% set x 10
10
% puts $x
10
% exit
```

2.2 Running a Script File

To execute a complete Tcl script file, use:

```
$ tclsh my_script.tcl
```

To run a script from inside another Tcl environment (like OpenROAD or Innovus):

```
% source my_script.tcl
```

2.3 Running Tcl Inside EDA Tools

Each EDA tool has its own Tcl interpreter. You can either type commands interactively or source a script:

```
# OpenROAD interactive mode
$ openroad
openroad> source design_setup.tcl

# Cadence Innovus
$ innovus -init my_flow.tcl

# Synopsys PrimeTime
$ pt_shell -f my_sta_script.tcl

# Yosys synthesis
$ yosys -s synthesis.tcl
```

2.4 Shebang Line for Direct Execution

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

You can make a Tcl script directly executable on Linux by adding a shebang line:

```
#!/usr/bin/env tclsh
puts "This script runs directly!"
```

```
$ chmod +x my_script.tcl
$ ./my_script.tcl
This script runs directly!
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

3. Tcl Basics

3.1 Variables

In Tcl, variables are created with the set command. To retrieve a value, prefix the variable name with $:

```
# Setting a variable
set design_name "my_chip"
set frequency  500

# Reading a variable
puts $design_name
puts $frequency
```

```
# Real VLSI example
set PDK        "sky130A"
set TOP_MODULE "dpram_512x8"
set CLK_PERIOD 10.0

puts "Running synthesis for $TOP_MODULE in $PDK PDK"
puts "Target clock period: ${CLK_PERIOD}ns"
```

3.2 Expressions

The expr command evaluates mathematical and logical expressions:

```
set a 100
set b 200
set sum  [expr {$a + $b}]
set diff [expr {$b - $a}]
set area [expr {$a * $b}]

puts "Sum  = $sum"
puts "Diff = $diff"
puts "Area = $area"
```

VLSI example – calculating core utilization:

```
set die_area  10000.0
set cell_area  6800.0
set utilization [expr {($cell_area / $die_area) * 100.0}]
puts [format "Core Utilization: %.2f%%" $utilization]
# Output: Core Utilization: 68.00%
```

3.3 Comments

Lines starting with # are comments. Inline comments use # after a semicolon:

```
# This is a full-line comment
set clk_freq 1000  ;# This is an inline comment (1 GHz)
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

3.4 The puts Command

puts prints text to the console. It automatically adds a newline. Use -nonewline to suppress it:

```
puts "Starting physical design flow..."
puts -nonewline "Loading: "
puts "design.def"

# Writing to a file handle
set fh [open "report.txt" w]
puts $fh "Timing Report Generated"
close $fh
```

3.5 Substitution Rules

Tcl has three types of substitution: command, variable, and backslash.

```
# Variable substitution with $
set name "OpenROAD"
puts "Tool: $name"

# Command substitution with []
set date [clock format [clock seconds] -format "%Y-%m-%d"]
puts "Date: $date"

# Double quotes allow substitution
puts "Design: $name, Date: $date"

# Braces prevent substitution
puts {Design: $name, Date: $date}
# Output: Design: \$name, Date: \$date
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

4. Control Structures

4.1 if Statement

The if command evaluates a condition and executes a block if true:

```
# Syntax: if {condition} { body }
set wns -0.5
if {$wns < 0} {
    puts "WARNING: Timing violation detected! WNS = $wns ns"
}
```

4.2 if-else Statement

```
set utilization 72
if {$utilization > 80} {
    puts "ERROR: Core utilization too high: $utilization%"
} else {
    puts "OK: Core utilization is $utilization%"
}
```

4.3 if-elseif-else Chain

```
set slack -1.2

if {$slack < -2.0} {
    puts "CRITICAL: Major timing violation (slack = $slack ns)"
} elseif {$slack < 0.0} {
    puts "WARNING: Minor timing violation (slack = $slack ns)"
} elseif {$slack < 0.5} {
    puts "CAUTION: Marginally passing (slack = $slack ns)"
} else {
    puts "PASS: Timing meets target (slack = $slack ns)"
}
```

4.4 switch Statement

The switch command handles multiple conditions cleanly:

```
set tool "openroad"

switch $tool {
    "innovus"  { puts "Running Cadence Innovus flow" }
    "icc2"     { puts "Running Synopsys ICC2 flow"   }
    "openroad" { puts "Running OpenROAD flow"        }
    default    { puts "Unknown tool: $tool"           }
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

4.5 Real VLSI Example – Checking PDK

```
set pdk "sky130A"

switch -exact $pdk {
    "sky130A"  { puts "SkyWater 130nm PDK (Version A) loaded" }
    "sky130B"  { puts "SkyWater 130nm PDK (Version B) loaded" }
    "GF180MCU" { puts "GlobalFoundries 180nm PDK loaded"      }
    "NanGate45" { puts "NanGate 45nm FreePDK loaded"          }
    default    { error "Unsupported PDK: $pdk"                  }
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

5. Loops

5.1 for Loop

The for loop is similar to C. It has an init, condition, step, and body:

```
# Syntax: for {init} {condition} {step} { body }
for {set i 1} {$i <= 5} {incr i} {
    puts "Iteration: $i"
}
```

VLSI example – sweeping clock frequencies:

```
# Sweep clock period from 5ns to 10ns in 1ns steps
for {set period 5} {$period <= 10} {incr period} {
    puts "Analyzing timing for clock period: ${period}ns"
    # In real tool: create_clock -period $period [get_ports clk]
}
```

5.2 foreach Loop

foreach iterates over a list. It is the most commonly used loop in VLSI scripts:

```
# Basic foreach
set modules {alu register_file multiplier cache}

foreach module $modules {
    puts "Processing module: $module"
}
```

Real OpenLane/OpenROAD example – loading all DEF files:

```
set def_files [glob -nocomplain *.def]

foreach def $def_files {
    puts "Found DEF file: $def"
    # read_def $def
}
```

foreach with multiple variables:

```
set port_list  {clk rst data_in data_out}
set directions {input input input output}

foreach port $port_list dir $directions {
    puts "Port: [format %-12s $port]  Direction: $dir"
}
```

5.3 while Loop

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
# Wait for a log file to appear (polling loop)
set max_wait 30
set wait_count 0

while {![file exists "synthesis.log"] && $wait_count < $max_wait} {
    puts "Waiting for synthesis to complete... ($wait_count s)"
    after 1000  ;# wait 1 second
    incr wait_count
}

if {[file exists "synthesis.log"]} {
    puts "Synthesis complete!"
} else {
    puts "ERROR: Synthesis timed out after $max_wait seconds"
}
```

5.4 Loop Control: break and continue

```
set violations {-0.1 -0.5 -2.0 0.3 -0.8}

foreach v $violations {
    if {$v > 0} { continue }           ;# skip passing paths
    if {$v < -1.5} {
        puts "CRITICAL violation: $v ns -- stopping check"
        break                          ;# stop on critical
    }
    puts "Violation: $v ns"
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

6. Procedures (procs)

6.1 Defining a Procedure

Procedures (procs) are reusable blocks of code that accept arguments and return values:

```
# Syntax: proc name {args} { body }
proc add {a b} {
    return [expr {$a + $b}]
}

puts [add 10 20]   ;# Output: 30
puts [add 5.5 4.5] ;# Output: 10.0
```

6.2 Procedures with Default Arguments

```
proc report_timing {design {format "detailed"} {max_paths 10}} {
    puts "Design:   $design"
    puts "Format:   $format"
    puts "MaxPaths: $max_paths"
}

report_timing "dpram"                ;# uses defaults
report_timing "dpram" "summary" 5   ;# overrides defaults
```

6.3 Variable Scope – global and upvar

```
set g_design_name "my_chip"  ;# Global variable

proc show_design {} {
    global g_design_name         ;# Bring global into scope
    puts "Global design: $g_design_name"
}

proc modify_by_ref {varname} {
    upvar 1 $varname local_var   ;# Reference to caller's var
    set local_var "modified_value"
}

show_design
set myvar "original"
modify_by_ref myvar
puts $myvar   ;# Output: modified_value
```

6.4 Real VLSI Procedure Examples

Procedure to calculate and report core utilization:

```
proc calc_utilization {die_area cell_area} {
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
    if {$die_area <= 0} {
        error "Die area must be positive"
    }
    set util [expr {($cell_area / $die_area) * 100.0}]
    set status [expr {$util > 80 ? "WARNING" : "OK"}]
    puts "$status: Core Utilization = [format %.2f $util]%"
    return $util
}

set u [calc_utilization 10000 6800]
puts "Returned utilization: $u"
```

Procedure to check if required files exist before running flow:

```
proc check_required_files {file_list} {
    set missing 0
    foreach f $file_list {
        if {![file exists $f]} {
            puts "ERROR: Missing required file: $f"
            incr missing
        } else {
            puts "OK:    Found file: $f"
        }
    }
    return $missing
}

set required {design.v constraints.sdc config.tcl}
set errors [check_required_files $required]
if {$errors > 0} {
    error "Cannot start flow: $errors file(s) missing"
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

7. Lists and Arrays

7.1 Creating Lists

```
# Simple list creation
set cells [list AND2X1 OR2X1 INVX1 NAND2X1 DFF]

# Or use braces
set layers {M1 M2 M3 M4 M5 M6 M7 M8 M9}

# List length
puts "Number of layers: [llength $layers]"
```

7.2 List Operations

```
set nets {clk rst_n data_in data_out vdd vss}

# Access elements (0-indexed)
puts [lindex $nets 0]      ;# clk
puts [lindex $nets end]    ;# vss

# Append to list
lappend nets scan_en
puts $nets

# Slice a sub-list
puts [lrange $nets 0 2]    ;# clk rst_n data_in

# Sort
puts [lsort $nets]

# Search
set idx [lsearch $nets clk]
puts "clk is at index: $idx"

# Check membership
if {[lsearch $nets "data_in"] >= 0} {
    puts "data_in net found in list"
}
```

7.3 List Iteration

```
set verilog_files {alu.v control.v datapath.v top.v}

foreach f $verilog_files {
    puts "Reading RTL file: $f"
    # yosys command: read_verilog $f
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

7.4 Arrays

Tcl arrays are associative (key-value pairs), similar to dictionaries:

```
# Setting array elements
set layer_info(M1,min_width)  0.12
set layer_info(M1,min_space)  0.12
set layer_info(M2,min_width)  0.14
set layer_info(M2,min_space)  0.14

# Reading
puts "M1 min width: $layer_info(M1,min_width) um"

# Iterating over array
foreach key [array names layer_info] {
    puts "$key = $layer_info($key)"
}

# Check key existence
if {[info exists layer_info(M3,min_width)]} {
    puts "M3 data exists"
} else {
    puts "M3 data not defined"
}
```

7.5 Dictionaries (Tcl 8.5+)

```
# Modern Tcl dictionaries
set design_cfg [dict create \
    top_module  dpram_512x8 \
    clock_port  clk \
    clk_period  10.0 \
    utilization 65 \
]

puts [dict get $design_cfg top_module]
dict set design_cfg utilization 70

dict for {key val} $design_cfg {
    puts "  $key => $val"
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

8. String Operations

8.1 String Length and Index

```
set module_name "DPRAM_512x8_v1"

puts [string length $module_name]     ;# 15
puts [string index  $module_name 0]   ;# D
puts [string range  $module_name 0 4] ;# DPRAM
```

8.2 Case Conversion

```
set pdk_name "Sky130A"
puts [string toupper $pdk_name]       ;# SKY130A
puts [string tolower $pdk_name]       ;# sky130a
```

8.3 String Matching (glob patterns)

```
set filename "design_top_v2.v"

if {[string match "*.v" $filename]} {
    puts "$filename is a Verilog file"
}

if {[string match "design_*" $filename]} {
    puts "$filename is a design file"
}
```

8.4 String Search

```
set log_line "Error: Timing violation in path: clk->data_out"

if {[string first "Error" $log_line] >= 0} {
    puts "Error found in log"
}

set idx [string first "clk" $log_line]
puts "clk appears at position: $idx"
```

8.5 String Replace and Trim

```
set path "  /home/vlsi/design/top.v  "

# Trim whitespace
set clean [string trim $path]
puts "'$clean'"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
# Replace
set new [string map {top.v top_v2.v} $path]
puts $new
```

8.6 format – Formatted Output

```
set wns  -0.123
set tns  -45.67
set whs   0.05

puts [format "WNS: %8.3f ns" $wns]
puts [format "TNS: %8.3f ns" $tns]
puts [format "WHS: %8.3f ns" $whs]

# Table-style formatting
puts [format "%-20s %10s %10s" "Path" "Slack" "Status"]
puts [format "%-20s %10.3f %10s" "clk->data_out" -0.123 "FAIL"]
puts [format "%-20s %10.3f %10s" "clk->addr_out"  0.456 "PASS"]
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

9. File Handling

9.1 Opening and Closing Files

```
# Open for writing  (w = write, r = read, a = append)
set fh [open "output.log" w]
puts $fh "Script started at [clock format [clock seconds]]"
close $fh

# Open for reading
set fh [open "output.log" r]
set content [read $fh]
close $fh
puts $content
```

9.2 Reading Line by Line

Reading files line-by-line is the most common approach for parsing EDA reports:

```
proc parse_report {filename} {
    if {![file exists $filename]} {
        puts "ERROR: File not found: $filename"
        return
    }
    set fh [open $filename r]
    set line_num 0
    while {[gets $fh line] >= 0} {
        incr line_num
        puts "Line $line_num: $line"
    }
    close $fh
    puts "Total lines: $line_num"
}

parse_report "synthesis.log"
```

9.3 Parsing a Synthesis Report

This is a common real-world task – extracting data from tool output:

```
proc extract_area_from_report {report_file} {
    set total_area 0.0
    set fh [open $report_file r]

    while {[gets $fh line] >= 0} {
        # Look for area line like: "Total cell area: 15234.56"
        if {[regexp {Total cell area:\s+([0-9.]+)} $line match area]} {
            set total_area $area
            puts "Found total cell area: $total_area um2"
        }
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
        # Look for timing: "Worst Negative Slack: -0.123"
        if {[regexp {Worst Negative Slack:\s+(-?[0-9.]+)} $line match wns]} {
            puts "WNS: $wns ns"
        }
    }
    close $fh
    return $total_area
}
```

9.4 Writing Reports

```
proc write_summary_report {design wns tns util outfile} {
    set fh [open $outfile w]
    puts $fh "======================================"
    puts $fh "  DESIGN SUMMARY REPORT"
    puts $fh "======================================"
    puts $fh [format "Design     : %s" $design]
    puts $fh [format "WNS        : %.3f ns" $wns]
    puts $fh [format "TNS        : %.3f ns" $tns]
    puts $fh [format "Utilization: %.2f%%" $util]
    set status [expr {$wns >= 0 ? "PASS" : "FAIL"}]
    puts $fh [format "Status     : %s" $status]
    puts $fh "======================================"
    close $fh
    puts "Report written to: $outfile"
}

write_summary_report "my_chip" -0.05 -12.3 68.5 "summary.rpt"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

10. Directory and File Operations

10.1 File Tests

```
# Check if file exists
if {[file exists "design.v"]} { puts "design.v found" }

# Check if it is a directory
if {[file isdirectory "results"]} { puts "results/ exists" }

# File size
set sz [file size "big_design.def"]
puts "File size: $sz bytes"

# File extension
puts [file extension "top_module.v"]   ;# .v
puts [file rootname "top_module.v"]    ;# top_module
puts [file tail "/home/vlsi/design/top.v"] ;# top.v
```

10.2 Creating Directories

```
proc setup_project_dirs {project_name} {
    set dirs [list \
        $project_name \
        $project_name/rtl \
        $project_name/constraints \
        $project_name/scripts \
        $project_name/results \
        $project_name/reports \
        $project_name/logs \
    ]
    foreach d $dirs {
        if {![file isdirectory $d]} {
            file mkdir $d
            puts "Created: $d"
        } else {
            puts "Exists:  $d"
        }
    }
}

setup_project_dirs "dpram_design"
```

10.3 Collecting Files with glob

```
# Get all Verilog files in current directory
set vfiles [glob -nocomplain *.v]
puts "Found [llength $vfiles] Verilog files:"
foreach f $vfiles { puts "  $f" }
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
# Recursive glob (Tcl 8.5+)
set all_v [glob -nocomplain -directory . -type f *.v]

# Get all DEF and LEF files
set pnr_files [glob -nocomplain *.def *.lef]
foreach f $pnr_files { puts $f }
```

10.4 File Copy, Move, Delete

```
# Copy a file
file copy -force design.v design_backup.v

# Rename/move
file rename results/timing.rpt results/timing_v1.rpt

# Delete
file delete -force temp_file.tmp

# Delete directory tree
file delete -force temp_work/
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

11. Environment Variables

11.1 Accessing Environment Variables

Tcl accesses system environment variables through the global $::env array:

```
# Access an environment variable
puts $::env(HOME)
puts $::env(PATH)
puts $::env(USER)
```

11.2 PDK and Tool Configuration

```
# Check PDK_ROOT (used in OpenLane/open_pdks)
if {[info exists ::env(PDK_ROOT)]} {
    puts "PDK Root: $::env(PDK_ROOT)"
} else {
    puts "ERROR: PDK_ROOT is not set!"
    puts "Run: export PDK_ROOT=/path/to/open_pdks"
    exit 1
}

# Construct full PDK path
set pdk     "sky130A"
set std_lib "sky130_fd_sc_hd"
set lib_path "$::env(PDK_ROOT)/$pdk/libs.ref/$std_lib"
puts "Library path: $lib_path"
```

11.3 Setting Environment Variables from Tcl

```
# Set/modify an environment variable
set ::env(DESIGN_NAME) "dpram_512x8"
set ::env(CLK_PERIOD)  "10.0"
set ::env(UTIL)        "65"

# Verify
puts "Design: $::env(DESIGN_NAME)"
puts "Period: $::env(CLK_PERIOD) ns"
```

11.4 OpenLane Configuration via env

```
# Typical OpenLane config.tcl uses env variables
set ::env(DESIGN_NAME)          "my_chip"
set ::env(VERILOG_FILES)        "[glob $::env(DESIGN_DIR)/src/*.v]"
set ::env(CLOCK_PORT)           "clk"
set ::env(CLOCK_NET)            "clk"
set ::env(CLOCK_PERIOD)         "10"
set ::env(FP_CORE_UTIL)         "65"
set ::env(FP_ASPECT_RATIO)      "1"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
set ::env(SYNTH_STRATEGY)       "AREA 0"
set ::env(PL_TARGET_DENSITY)    "0.7"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

12. Regular Expressions

12.1 regexp – Pattern Matching

```
# Basic match
set line "WNS: -0.234 ns"
if {[regexp {WNS:\s+(-?[0-9.]+)} $line]} {
    puts "Line contains WNS"
}

# Capture groups
if {[regexp {WNS:\s+(-?[0-9.]+)} $line match wns_val]} {
    puts "WNS value: $wns_val"
}
```

12.2 Extracting Timing Violations

```
proc parse_timing_report {filename} {
    set violations {}
    set fh [open $filename r]

    while {[gets $fh line] >= 0} {
        # Match: slack (MET) or slack (VIOLATED): -0.123
        if {[regexp {slack.*?:\s+(-?[0-9]+\.[0-9]+)} $line m slack]} {
            if {$slack < 0} {
                lappend violations $slack
                puts "VIOLATION: slack = $slack ns"
            }
        }
    }
    close $fh
    puts "Total violations: [llength $violations]"
    return $violations
}
```

12.3 regsub – Substitution

```
# Replace text using regex
set path "/home/user/VLSI/design_v1/top.v"

# Replace version
regsub {v1} $path {v2} new_path
puts $new_path

# Remove comments from a line
set code_line "set x 10 ;# This is a comment"
regsub {;#.*$} $code_line {} clean_line
puts "Clean: '$clean_line'"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
# Replace all spaces with underscores
regsub -all {\s+} "my design name" {_} result
puts $result   ;# my_design_name
```

12.4 Common Regex Patterns for VLSI

Pattern	Matches
-?[0-9]+\.[0-9]+	Floating point numbers (timing values)
[0-9]+\s*ns	Time values ending in ns
M[1-9][0-9]?	Metal layer names (M1, M2, M9)
[A-Z_][A-Z0-9_]+	UPPER_CASE signal/port names
slack.*?VIOLATED	Timing violation line in STA
Total\s+area:\s+([0-9.]+)	Area extraction from reports

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

13. Tcl in ASIC Design Flows

13.1 Synthesis with Yosys

Yosys uses Tcl-style commands in synthesis scripts:

```
# yosys_synthesis.tcl
yosys read_verilog rtl/alu.v
yosys read_verilog rtl/control.v
yosys read_verilog rtl/top.v

yosys synth -top my_chip -flatten
yosys dfflibmap -liberty pdk/sky130_fd_sc_hd.lib
yosys abc -liberty pdk/sky130_fd_sc_hd.lib

yosys write_verilog -noattr results/netlist.v
yosys stat
```

13.2 OpenROAD Physical Design Flow

```
# openroad_flow.tcl
read_lef pdk/sky130_fd_sc_hd.tlef
read_lef pdk/sky130_fd_sc_hd.lef
read_liberty pdk/sky130_fd_sc_hd_tt_025C_1v80.lib
read_verilog results/netlist.v
link_design my_chip

# Floor planning
initialize_floorplan -utilization 65 \
    -aspect_ratio 1.0 \
    -core_space 10

# IO placement
place_pins -hor_layers M4 -ver_layers M3

# Power planning
add_global_connection -net VDD -pin_pattern VPB -power
add_global_connection -net VSS -pin_pattern VNB -ground
global_connect

# Placement
global_placement -density 0.7
legalize_placement
detailed_placement

# CTS
clock_tree_synthesis -root_buf CLKBUF_X3 -buf_list {CLKBUF_X1 CLKBUF_X3}

# Routing
route_guides_check
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
detailed_route

# Reports
report_design_area
report_timing
```

13.3 STA with PrimeTime (Synopsys)

```
# sta_analysis.tcl - PrimeTime STA script
set_app_options -name time.disable_recovery_removal_checks -value false

# Read libraries and netlist
read_lib /pdk/tt_025C_1v80.db
read_verilog results/final_netlist.v
link_design my_chip

# Apply constraints
read_sdc constraints/my_chip.sdc

# Apply parasitics
read_parasitics -format spef results/my_chip.spef

# Run analysis
update_timing

# Reports
report_timing -max_paths 10 > reports/timing_setup.rpt
report_constraint -all_violators > reports/violations.rpt
report_power > reports/power.rpt
```

13.4 Cadence Innovus Flow

```
# innovus_flow.tcl
# Initialize design
init_design

# Floor plan
floorPlan -su 1.0 0.70 2.0 2.0 2.0 2.0

# Power planning
globalNetConnect VDD -type pgpin -pin VDD -inst *
globalNetConnect VSS -type pgpin -pin VSS -inst *
sroute -nets {VDD VSS}

# Placement
place_design
optDesign -preCTS

# Clock tree
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
create_clock_tree_spec -output cts.spec -bufFootprint {CLKBUF1 CLKBUF2}
clockDesign -specFile cts.spec -outDir cts_results
optDesign -postCTS -hold

# Routing
routeDesign -globalDetail
optDesign -postRoute
optDesign -postRoute -hold

# Signoff
verifyGeometry
verifyConnectivity
streamOut final.gds -mapFile gds_map.txt
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

14. Practical VLSI Tcl Scripts

14.1 RTL File Collector

This script recursively finds all Verilog and SystemVerilog files in a directory tree and writes them to a filelist:

```
#!/usr/bin/env tclsh
# collect_rtl.tcl - Collect all RTL files into a filelist

proc collect_rtl_files {base_dir extensions outfile} {
    set all_files {}
    foreach ext $extensions {
        set found [glob -nocomplain -directory $base_dir $ext]
        set all_files [concat $all_files $found]
    }

    set fh [open $outfile w]
    puts $fh "# RTL Filelist - Generated by collect_rtl.tcl"
    puts $fh "# Generated: [clock format [clock seconds]]"
    puts $fh "# Directory: $base_dir"
    puts $fh ""

    foreach f [lsort $all_files] {
        puts $fh $f
        puts "Added: $f"
    }
    close $fh
    puts "\nTotal files: [llength $all_files]"
    puts "Filelist: $outfile"
}

collect_rtl_files "./rtl" {*.v *.sv *.vh} "rtl_filelist.f"
```

14.2 Synthesis Log Parser

Parse a Yosys synthesis log and extract key metrics:

```
#!/usr/bin/env tclsh
# parse_synth_log.tcl

proc parse_yosys_log {logfile} {
    set cells 0
    set area  0
    set wires 0
    set fh [open $logfile r]

    while {[gets $fh line] >= 0} {
        if {[regexp {Number of cells:\s+(\d+)} $line m v]} { set cells $v }
        if {[regexp {Chip area for top module.*?:\s+([0-9.]+)} $line m v]} {
set area $v }
```

```
        if {[regexp {Number of wires:\s+(\d+)} $line m v]} { set wires $v }
    }
    close $fh

    puts "============================"
    puts "  SYNTHESIS SUMMARY"
    puts "============================"
    puts [format "  Cells : %8d"   $cells]
    puts [format "  Wires : %8d"   $wires]
    puts [format "  Area  : %8.2f um2" $area]
    puts "============================"
}

parse_yosys_log "synth.log"
```

14.3 Timing Report Analyzer

```
#!/usr/bin/env tclsh
# timing_analyzer.tcl

proc analyze_timing {rpt_file} {
    set wns 9999.0
    set tns 0.0
    set violating_paths 0
    set fh [open $rpt_file r]

    while {[gets $fh line] >= 0} {
        if {[regexp {slack.*?:\s+(-?[0-9]+\.[0-9]+)} $line m slack]} {
            set slack [expr {double($slack)}]
            if {$slack < $wns} { set wns $slack }
            if {$slack < 0} {
                set tns [expr {$tns + $slack}]
                incr violating_paths
            }
        }
    }
    close $fh

    puts "WNS: [format %.3f $wns] ns"
    puts "TNS: [format %.3f $tns] ns"
    puts "Violating paths: $violating_paths"

    if {$wns < 0} { puts "Status: TIMING FAILED" } \
    else           { puts "Status: TIMING PASSED" }
}

analyze_timing "timing.rpt"
```

14.4 Automatic Design Folder Creator

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
#!/usr/bin/env tclsh
# create_project.tcl

proc create_asic_project {name pdk} {
    puts "Creating ASIC project: $name ($pdk)"
    set base $name

    foreach dir {rtl constraints scripts results reports logs gds spice} {
        file mkdir "$base/$dir"
        puts "  [file mkdir $base/$dir; expr {"OK"}] $base/$dir"
    }

    # Write config template
    set cfg [open "$base/config.tcl" w]
    puts $cfg "set ::env(DESIGN_NAME)  \"$name\""
    puts $cfg "set ::env(PDK)          \"$pdk\""
    puts $cfg "set ::env(CLOCK_PORT)   \"clk\""
    puts $cfg "set ::env(CLOCK_PERIOD) \"10\""
    close $cfg
    puts "  Created: $base/config.tcl"
    puts "\nProject '$name' ready!"
}

create_asic_project "my_chip" "sky130A"
```

14.5 OpenLane Automation Script

```
#!/usr/bin/env tclsh
# run_openlane.tcl - Automate OpenLane flow stages

set DESIGN    "dpram_512x8"
set PDK       "sky130A"
set RUN_DIR   "runs/[clock format [clock seconds] -format %Y%m%d_%H%M%S]"

set stages {
    synthesis floorplan placement cts routing
    magic netgen
}

proc run_stage {stage design} {
    puts "\n>>> Running: $stage for $design"
    set cmd "flow.tcl -design $design -tag run_$stage -from $stage -to $stage"
    set rc [catch {exec bash -c $cmd} out]
    if {$rc != 0} {
        puts "ERROR in $stage: $out"
        return 0
    }
    puts "Completed: $stage"
    return 1
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
foreach stage $stages {
    if {![run_stage $stage $DESIGN]} {
        puts "Flow halted at: $stage"
        exit 1
    }
}
puts "\nAll stages completed successfully!"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

15. Advanced Tcl Concepts

15.1 Namespaces

Namespaces prevent name collisions in large scripts – essential for shared VLSI utility libraries:

```
namespace eval vlsi_utils {
    variable version 1.0

    proc calc_util {cell_area die_area} {
        return [expr {($cell_area / $die_area) * 100.0}]
    }

    proc check_timing {wns} {
        return [expr {$wns >= 0 ? "PASS" : "FAIL"}]
    }
}

# Call namespace procedure
set u [vlsi_utils::calc_util 6800 10000]
puts "Utilization: [format %.2f $u]%"

# Import into current scope
namespace import vlsi_utils::*
puts [check_timing -0.5]   ;# FAIL
```

15.2 Error Handling with catch

```
# catch: returns 0 on success, 1 on error
set rc [catch {
    set fh [open "nonexistent_file.rpt" r]
} err_msg]

if {$rc != 0} {
    puts "ERROR caught: $err_msg"
} else {
    puts "File opened OK"
}

# Using error to raise exceptions
proc load_pdk {pdk_path} {
    if {![file isdirectory $pdk_path]} {
        error "PDK directory not found: $pdk_path" "" 101
    }
    puts "PDK loaded from: $pdk_path"
}

if {[catch {load_pdk "/invalid/path"} msg opts]} {
    puts "Flow error: $msg"
    puts "Error code: [dict get $opts -errorcode]"
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
}
```

15.3 Packages

```
# Create a reusable VLSI utilities package
# File: vlsi_utils/pkgIndex.tcl
package ifneeded vlsi_utils 1.0 {
    source [file join [file dirname [info script]] vlsi_utils.tcl]
}

# In your main script
lappend auto_path ./vlsi_utils
package require vlsi_utils 1.0

# Now use it
vlsi_utils::parse_report timing.rpt
```

15.4 exec – Running System Commands

```
# Execute a shell command and capture output
set git_log [exec git log --oneline -5]
puts $git_log

# Run a tool and check for errors
if {[catch {exec yosys -s synth.tcl} result]} {
    puts "Synthesis failed!"
    puts $result
    exit 1
} else {
    puts "Synthesis OK"
}
```

15.5 Callbacks and apply

```
# Lambda-like functions with apply
set double [list apply {{x} {expr {$x * 2}}}]
puts [{*}$double 21]   ;# 42

# Callbacks example
proc on_each_file {filelist callback} {
    foreach f $filelist {
        {*}$callback $f
    }
}

on_each_file [glob *.v] {apply {{f} {puts "Checking: $f"}}}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

16. Example Mini Projects

16.1 Project 1 – Verilog Module Scanner

Scans a directory and prints all module definitions found in Verilog files:

```
#!/usr/bin/env tclsh
# scan_modules.tcl

proc scan_verilog_modules {search_dir} {
    set v_files [glob -nocomplain -directory $search_dir *.v *.sv]

    if {[llength $v_files] == 0} {
        puts "No Verilog files found in: $search_dir"
        return
    }

    set total_mods 0
    foreach vfile $v_files {
        set fh [open $vfile r]
        set file_mods {}
        while {[gets $fh line] >= 0} {
            if {[regexp {^\s*module\s+(\w+)} $line m modname]} {
                lappend file_mods $modname
                incr total_mods
            }
        }
        close $fh
        if {[llength $file_mods] > 0} {
            puts "[file tail $vfile]:"
            foreach m $file_mods { puts "  -> module: $m" }
        }
    }
    puts "\nTotal modules found: $total_mods"
}

scan_verilog_modules "./rtl"
```

16.2 Project 2 – Design File Checker

Verifies that all required files for an ASIC flow are present:

```
#!/usr/bin/env tclsh
# check_design_files.tcl

proc check_design_ready {design_dir} {
    set required_files [list \
        "$design_dir/config.tcl" \
        "$design_dir/constraints/design.sdc" \
    ]
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
    set required_dirs [list \
        "$design_dir/rtl" \
        "$design_dir/results" \
    ]

    set errors 0

    puts "Checking files:"
    foreach f $required_files {
        if {[file exists $f]} {
            puts "  [OK]  $f"
        } else {
            puts "  [!!]  MISSING: $f"
            incr errors
        }
    }

    puts "\nChecking directories:"
    foreach d $required_dirs {
        if {[file isdirectory $d]} {
            set cnt [llength [glob -nocomplain -directory $d *.v *.sv]]
            puts "  [OK]  $d ($cnt RTL files)"
        } else {
            puts "  [!!]  MISSING DIR: $d"
            incr errors
        }
    }

    if {$errors == 0} {
        puts "\nDesign is READY to run!"
    } else {
        puts "\nFound $errors error(s). Please fix before running."
    }
    return $errors
}

check_design_ready "./my_chip"
```

16.3 Project 3 – Simple Design Report Generator

```
#!/usr/bin/env tclsh
# gen_report.tcl - Generate HTML design summary report

proc gen_html_report {design wns tns util power outfile} {
    set pass [expr {$wns >= 0}]
    set color [expr {$pass ? "#28a745" : "#dc3545"}]
    set status [expr {$pass ? "PASS" : "FAIL"}]
    set date [clock format [clock seconds]]

    set fh [open $outfile w]
    puts $fh "<html><body>"
```

```
    puts $fh "<h1>Design Report: $design</h1>"
    puts $fh "<p>Generated: $date</p>"
    puts $fh "<table border='1'>"
    puts $fh "  <tr><th>Metric</th><th>Value</th></tr>"
    puts $fh "  <tr><td>WNS</td><td>[format %.3f $wns] ns</td></tr>"
    puts $fh "  <tr><td>TNS</td><td>[format %.3f $tns] ns</td></tr>"
    puts $fh "  <tr><td>Utilization</td><td>[format %.1f $util]%</td></tr>"
    puts $fh "  <tr><td>Power</td><td>[format %.3f $power] mW</td></tr>"
    puts $fh "  <tr><td>Status</td>"
    puts $fh "    <td style='color:$color;font-weight:bold'>$status</td></tr>"
    puts $fh "</table></body></html>"
    close $fh
    puts "Report: $outfile"
}

gen_html_report "dpram_512x8" -0.02 -5.4 68.5 12.3 "report.html"
```

17. Best Practices for VLSI Tcl Scripts

17.1 Script Structure

```
#!/usr/bin/env tclsh
# ============================================================
# Script : run_pd_flow.tcl
# Purpose: Run complete physical design flow for $DESIGN
# Author : Bapuji Banothu
# Date   : 2026
# ============================================================

# 1. Global configuration
set DESIGN       "dpram_512x8"
set PDK          "sky130A"
set RESULTS_DIR  "results"

# 2. Source utility libraries
source scripts/vlsi_utils.tcl

# 3. Procedures
proc run_synthesis {} { ... }
proc run_floorplan  {} { ... }
proc run_routing    {} { ... }

# 4. Main execution
puts "Starting PD flow for $DESIGN"
run_synthesis
run_floorplan
run_routing
puts "Flow complete!"
```

17.2 Naming Conventions

Element	Convention & Example
Global variables	ALL_CAPS: DESIGN_NAME, CLK_PERIOD
Local variables	lowercase_snake: cell_area, net_count
Procedures (procs)	verb_noun: calc_util, read_report
Boolean flags	is_ or has_ prefix: is_verbose, has_error
File handles	fh suffix: log_fh, rpt_fh
Counters/indices	Short descriptive: i, j, idx, cnt

17.3 Debugging Techniques

```
# Debug mode flag
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
set DEBUG 1

proc dbg {msg} {
    global DEBUG
    if {$DEBUG} { puts "\[DEBUG\] $msg" }
}

dbg "Loading config file..."

# Inspect variable
puts "x = [expr {[info exists x] ? $x : "<undef>"}]"

# Trace variable changes
trace add variable my_var write {apply {{name el op} {
    upvar 1 $name v
    puts "Variable $name changed to: $v"
}}}

# Check proc argument count
proc safe_div {a b} {
    if {$b == 0} { error "Division by zero!" }
    return [expr {$a / $b}]
}
```

17.4 Performance Tips

- Use lappend instead of set list [concat $list $new] for list building
- Read large files with gets in a while loop, not read + split
- Pre-compile regex patterns with regexp -inline for repeated use
- Use dict instead of arrays for key-value data in Tcl 8.5+
- Avoid nested foreach over large lists – use arrays for O(1) lookup

 NOTE: In EDA tools, always test scripts on small designs first before running on large chips to avoid long runtimes with bugs.

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

18. Interview Questions – Tcl for VLSI Engineers

The following 20 questions are commonly asked in VLSI/EDA industry interviews. Study both the question and the detailed answer.

Q1. What is Tcl and why is it used in EDA tools?

Answer: Tcl (Tool Command Language) is a lightweight, embeddable scripting language. EDA vendors use it because it is easy to embed inside C/C++ tool engines, supports interactive sessions, and has excellent string processing – critical for parsing log files and reports. Tools like Cadence Innovus, Synopsys PrimeTime, OpenROAD, and Yosys all use Tcl as their primary scripting interface.

Q2. What is the difference between {} and "" in Tcl?

Answer: Double quotes ("") allow variable and command substitution to occur ($var, [cmd]). Curly braces ({}) prevent all substitution – the text is treated literally. For expressions in if/while/for, always use {} to avoid double substitution, e.g., if {$x > 0} is correct; if "$x > 0" can cause errors.

Q3. How do you read a file line-by-line in Tcl?

```
set fh [open "file.txt" r]
while {[gets $fh line] >= 0} {
    puts $line
}
close $fh
```

Q4. What is the difference between set and variable?

Answer: set creates or assigns a variable in the current scope. variable is used inside a namespace to declare namespace-level variables. Inside a proc, you must use global to access globals, or upvar to reference the caller's variables.

Q5. How does catch work in Tcl?

Answer: catch runs a script and returns 0 if successful, or a non-zero error code if it fails. The second argument captures the error message. This is used for safe file operations, tool command execution, and flow error handling.

Q6. What is lsearch and how is it used?

```
set cells {AND2X1 OR2X1 INVX1}
set idx [lsearch $cells OR2X1]
# Returns 1 (0-indexed). Returns -1 if not found.
if {$idx >= 0} { puts "Found at index $idx" }
```

Q7. How do you pass a variable by reference in Tcl?

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

Answer: Use upvar. It creates a local alias to a variable in a calling scope. This allows procs to modify the caller's variables, similar to passing a pointer in C.

```
proc increment {varname} {
    upvar 1 $varname v
    incr v
}
set count 5
increment count
puts $count   ;# 6
```

Q8. What does the glob command do?

Answer: glob returns a list of files matching a pattern, similar to shell globbing. It supports wildcards (* ? []) and is used extensively to collect RTL files, DEF files, and reports in VLSI scripts. The -nocomplain flag prevents errors when no files match.

Q9. How do you extract a number from a string using regexp?

```
set line "WNS: -0.234 ns"
if {[regexp {(-?[0-9]+\.[0-9]+)} $line match val]} {
    puts "Extracted: $val"  ;# -0.234
}
```

Q10. What is the difference between expr and set?

Answer: set stores a value. expr evaluates a mathematical or boolean expression and returns the result. You must use expr for any arithmetic: set result [expr {\$a + \$b}].

Q11. How do you define and call a procedure with default arguments?

```
proc greet {name {greeting "Hello"}} {
    puts "$greeting, $name!"
}
greet "Bapuji"          ;# Hello, Bapuji!
greet "Bapuji" "Namaste";# Namaste, Bapuji!
```

Q12. How do you use Tcl arrays in VLSI scripts?

```
set pin_load(clk)     0.05
set pin_load(data_in) 0.10
foreach pin [array names pin_load] {
    puts "$pin: $pin_load($pin) pF"
}
```

Q13. What is the purpose of namespaces in Tcl?

Answer: Namespaces prevent naming conflicts when combining multiple utility scripts. They create isolated scopes for variables and procedures. Example: vlsi_utils::calc_util is distinct from sta_utils::calc_util even if both have the same proc name.

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

Q14. How do you write to a file in Tcl?

```
set fh [open "report.txt" w]
puts $fh "Total Violations: 3"
close $fh
```

Q15. What is the incr command?

Answer: incr increments an integer variable by 1 (default) or by a specified step: incr counter; incr counter 5; incr counter -1. It is faster than set x [expr {\$x + 1}] and cleaner for loop counters.

Q16. What is source in Tcl?

Answer: source executes another Tcl file in the current interpreter scope. It is used to load utility libraries, PDK configurations, and flow scripts. In tools like OpenROAD and Innovus, source is the standard way to run automation scripts.

Q17. Explain the difference between lindex and lrange.

```
set lst {a b c d e}
puts [lindex $lst 2]      ;# c  (single element)
puts [lrange $lst 1 3]    ;# b c d (sub-list, inclusive)
puts [lindex $lst end]    ;# e (last element)
```

Q18. What is format used for in Tcl?

Answer: format produces formatted strings, similar to printf in C. It is essential for creating aligned tables in reports and controlling decimal precision for timing values.

```
puts [format "%-20s %8.3f ns" "clk->q" -0.123]
;# clk->q               -0.123 ns
```

Q19. How do you check if a variable is defined?

```
if {[info exists my_var]} {
    puts "my_var = $my_var"
} else {
    puts "my_var is not defined"
}
```

Q20. How does Tcl handle error propagation in ASIC flows?

Answer: EDA tools return non-zero error codes when commands fail. Use catch to intercept errors, log them, and decide whether to abort or continue. Use error to raise custom exceptions in your procs. Always validate file existence and PDK paths before starting any flow stage.

```
proc safe_read_def {filename} {
    if {![file exists $filename]} {
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

```
        error "DEF file not found: $filename"
    }
    read_def $filename
    puts "Loaded: $filename"
}

if {[catch {safe_read_def "final.def"} msg]} {
    puts "Flow aborted: $msg"
    exit 1
}
```

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

19. Conclusion

19.1 What You Have Learned

This document has taken you on a complete journey through Tcl scripting as it applies to VLSI Physical Design. You started with the absolute basics – variables, puts, and expressions – and progressed to advanced topics like namespaces, error handling, and packages.

Along the way, every concept was illustrated with real VLSI examples drawn from actual EDA tool usage: OpenROAD floor planning commands, Yosys synthesis scripts, OpenLane configuration files, Cadence Innovus flow automation, and Synopsys PrimeTime STA scripting.

19.2 The ASIC Flow Automation Advantage

Manual EDA tool operation is error-prone and slow. Tcl scripting transforms repetitive, multi-step ASIC flows into reliable, automated pipelines. The benefits are significant:

- Reproducibility – the same script produces the same result every time
- Speed – multi-day manual flows become multi-hour automated runs
- Error detection – scripts can parse logs, detect failures, and alert the designer
- Integration – Tcl glues together synthesis, PnR, STA, and verification tools
- Customization – scripts can be parameterized for different designs and PDKs

19.3 Key Takeaways

Topic	Key Takeaway
Variables & Expressions	Use set and expr. Always wrap expr in {}
Control Flow	if/switch for branching; for/foreach/while for loops
Procedures	Modular code with proc; use upvar for by-reference args
Lists & Arrays	Lists for ordered data; arrays and dicts for key-value
File Handling	open/gets/puts/close; always check file existence first
Regex	regexp/regsub for parsing reports and extracting values
Error Handling	catch all external calls; error to propagate failures
Namespaces	Organize utilities into namespaces for large projects
Best Practices	Comment, structure, debug, and name variables consistently

19.4 Next Steps

To continue your Tcl mastery in VLSI:

1. Practice by writing scripts for your own designs – start small

© 2026 VLSI Bapuji Banothu M.Tech VLSI SD – All Rights Reserved

2. Study the Tcl scripts shipped with OpenLane (scripts/ directory)
3. Read the OpenROAD Tcl API documentation
4. Explore the Tk GUI extension for building design monitoring dashboards
5. Learn how to interface Tcl with Python using tkinter or subprocess
6. Study PDK Tcl scripts in sky130A/libs.tech/ for real-world patterns

> **NOTE:** The best way to learn Tcl is to write scripts every day. Start by automating one small task in your current design flow, then grow from there.

Happy Scripting & Happy Taping Out!

© 2026 Bapuji Banothu | M.Tech VLSI SD